

Chapter 5

Functions: Turn Repeated Biology Code into Tools

Create your own reusable commands with parameters, return values, and clean design.

EDOS learning idea

We will use biology and DNA examples because they are concrete, visual, and easy to experiment with. Type the examples yourself, change one thing, run again, and observe what changed.

What you will be able to do

- Explain what a function is.
- Define functions with `def`.
- Pass information using parameters.
- Return results with `return`.
- Build reusable DNA helper functions.

Classroom rhythm

Read a small idea -> run the example -> change one value -> predict the result -> run again -> explain what happened in your own words.

5.1 Why functions?

If you calculate GC percentage five times by copying the same code, you now have five places to fix if the formula changes. A function lets you write the idea once and reuse it.

Think of a function as a small lab protocol: it has a name, it receives materials, it performs steps, and it gives you a result.

5.2 Define and call a function

```
def say_hello():  
    print('Hello from the biology lab!')  
  
say_hello()
```

`def` starts a function definition. The indented code is the function body. Nothing happens until you call the function.

5.3 Parameters: give the function data

```
def describe_sequence(dna):  
    print('Sequence:', dna)  
    print('Length:', len(dna))  
  
describe_sequence('ATGCGT')  
describe_sequence('GGGCCCAA')
```

`dna` is a parameter. Each call can provide a different sequence.

5.4 return: give a value back

```
def gc_percent(dna):  
    gc = dna.count('G') + dna.count('C')  
    return gc / len(dna) * 100  
  
result = gc_percent('ATGCGCGTAA')  
print(result)
```

`return` is different from `print()`. `print()` displays something, `return` gives a value back to the rest of your program so it can be stored, compared, plotted, or written to a file.

5.5 A small DNA toolkit

```
def clean_dna(dna):
    return dna.strip().upper()

def base_counts(dna):
    dna = clean_dna(dna)
    return {
        'A': dna.count('A'),
        'C': dna.count('C'),
        'G': dna.count('G'),
        'T': dna.count('T')
    }

print(base_counts(' atgcaa '))
```

Functions can call other functions. `clean_dna()` handles cleanup, while `base_counts()` focuses on counting. This separation makes code easier to test.

5.6 Default parameters and helpful checks

```
def label_sequence(dna, name='unknown'):
    dna = dna.strip().upper()
    if len(dna) == 0:
        return 'Empty sequence'
    return f'{name}: {len(dna)} bases'

print(label_sequence('ATGC'))
print(label_sequence('ATGCGT', 'sample_7'))
```

A default parameter gives a value when the caller does not provide one. You can also add simple checks before doing calculations.

Try it yourself

These are practice prompts, not a test. If your code breaks, that is useful information. Read the error, change one thing, and try again.

1. Write a function `nucleotide_count(dna, base)` that returns the count of one base.
2. Write a function `is_long(dna)` that returns `True` when length is at least 20.
3. Write a function `gc_percent(dna)` and test it on three sequences.
4. Write a function `clean_dna(dna)` that removes outer spaces and converts to uppercase.
5. Explain in one sentence why `return` is more flexible than `print()`.

Tiny debugging habit

When something fails, check spelling, quotation marks, brackets, colons, and indentation first. Then print intermediate values so you can see what Python sees.

Chapter summary

- Functions package reusable logic.
- Parameters are inputs.
- `return` produces a reusable result.
- Small focused functions are easier to test.
- A DNA toolkit can grow one function at a time.

Next step

Open Notebook 05 in EDOS Jupyter and run the matching exercises. Then continue to Chapter 6.

EDOS - Educational Desktop Operating Simulator
Copyright by Dr. Yash Munnalal Gupta